# NP-hardness of exact Hadamard decomposition with one rank bound equal to two

Working manuscript · Awaiting independent human expert review
15 September 2026

## Abstract
Exact Hadamard decomposition asks whether a rational matrix is the entrywise product of two real matrices with prescribed rank bounds. We give a deterministic polynomial-time reduction from 3-SAT, even when the input matrix is nonnegative integer and one rank bound is two. An identity submatrix forces a basis of product vectors. Additional rows synchronize two-choice selectors, and weighted rows around a cycle encode an exact subset-product equation. The converse holds for arbitrary real factors, without sign restrictions. A determinant predicate recovers a satisfying assignment from every valid decomposition. For witnesses encoded over a common real algebraic primitive element, a subresultant and Sturm procedure implements recovery in polynomial bit time. The construction addresses an explicitly stated complexity question; its novelty is supported within the literature coverage described here and remains subject to expert assessment.

## 1. Introduction
Hadamard decomposition represents a matrix by multiplying low-rank matrices entrywise. It can represent a matrix of rank equal to the product of the factor ranks. This expressive capacity motivates its use as a multiplicative model for matrix compression. Ciaperoni, Gionis and Mannila introduced a data-analysis formulation, placed exact real feasibility in the existential theory of the reals, and left its formal computational complexity open [1].

Subsequent work developed alternating and manifold-based numerical algorithms [2], [3]. The product-vector interpretation is also known: rank-one reshapes of row factors recover the Hadamard representation [3]. These algorithms and reformulations do not decide exact feasibility for all rational inputs. Our contribution is a reduction that forces discrete choices within the unrestricted real factorization model.

For a rational matrix $M \in \mathbb{Q}^{\mu \times N}$ and positive integer bounds $p, q \leq \min(\mu, N)$, define exact Hadamard feasibility by

$$M = (UV) \odot (WZ), \tag{1}$$

where $U \in \mathbb{R}^{\mu \times p}$, $V \in \mathbb{R}^{p \times N}$, $W \in \mathbb{R}^{\mu \times q}$ and $Z \in \mathbb{R}^{q \times N}$. The symbol $\odot$ denotes entrywise multiplication. Factor entries may have arbitrary signs; the two ordinary products have ranks at most their respective bounds.

**Theorem 1 (Main result).** Exact Hadamard feasibility is NP-hard under deterministic polynomial-time many-one reductions, even for nonnegative integer matrices with $p = 2$. More precisely, a 3-SAT formula with $n$ occurring variables and $m$ clauses maps to an instance with

$$K = \max(3, 2n + 2m), \quad \mu = K^2 + 9K, \quad N = 4K, \quad q = 2K. \tag{2}$$

The formula is satisfiable if and only if this instance is feasible. Every valid real decomposition determines a satisfying assignment by explicit determinant zero tests. On the common-primitive algebraic witness encoding defined in Section 6, these tests give a deterministic algorithm polynomial in the formula length plus the supplied witness length.

The source reduction uses the standard many-one framework [4]. A prime-exponent encoding first expresses satisfaction as subset product. The new matrix construction then makes each item correspond to one of two orientations of a product-vector selector. Calibration retains a nonzero continuous parameter at each selector. A cycle of weighted rows makes the product of the selected items equal

a prescribed target. The second rank bound grows with the source instance; the theorem does not fix both bounds.

## 2. From 3-SAT to subset product

Let $F$ be a conjunction of $m$ clauses, each containing three literals. Repeated literals are allowed. Number the occurring variables consecutively from 1 to $n$; the empty conjunction has $n = 0$. Renaming the occurring variables of an ordinary 3-SAT formula is polynomial-time. This convention also ensures that the assignment length is bounded by the source encoding length.

Choose distinct primes $\pi_i$ for variables and $\kappa_j$ for clauses. For each variable $i$ and truth value $b$, let $c_{i,b,j}$ be the number of occurrences in clause $j$ made true by that value. Create two assignment items per variable and two slack items per clause:

$$r(i,b) = \pi_i \prod_{j=1}^{m} \kappa_j^{c_{i,b,j}}, \quad \kappa_j, \quad \kappa_j^2. \tag{3}$$

The required product is

$$T = \left( \prod_{i=1}^{n} \pi_i \right) \left( \prod_{j=1}^{m} \kappa_j^4 \right). \tag{4}$$

Order the assignment items by variable, with the true item before the false item, followed by the clause slack pairs. Append ones if needed to obtain $K \geq 3$ items, indexed $r_0, \ldots, r_{K-1}$.

**Lemma 1 (Prime encoding).** A subset of these items has product $T$ if and only if $F$ is satisfiable. Reading the selected assignment items recovers a satisfying assignment from every exact-product subset.

**Proof.** The exponent of $\pi_i$ forces exactly one of variable $i$'s assignment items. Its selected truth value determines how many literals of each clause are true. For clause $j$, let this count be $c_j$, between zero and three. Its two slack items can add exponent zero, one, two or three. The required exponent four is reachable exactly when $c_j \geq 1$. This proves both implications, including repeated literals. Padding by ones changes no exponent. For the empty formula, $T = 1$, and the empty subset is feasible. □

## 3. Matrix construction

We now take any positive integers $r_0, \ldots, r_{K-1}, T$ with $K \geq 3$. For each item $i$, reserve four column indices $L_i, R_i, X_i, Y_i$, in that order. Let $e_J$ denote the unit row vector at index $J$ in $\mathbb{R}^{4K}$. The matrix begins with the identity $I_{4K}$. The remaining rows are prescribed coefficient vectors relative to these indices.

First add $e_{L_i} + e_{L_j}$ for every $i < j$, then the corresponding $R$ rows. For each $i$, add the four rows pairing either $X_i$ or $Y_i$ with either $L_i$ or $R_i$. Next add one calibration row per item:

$$C_i = e_{L_i} + e_{R_i} + e_{X_i} + e_{Y_i}. \tag{5}$$

Finally add one cycle row per item. A cycle row has only two nonzero blocks, each of length four. Row $i$ uses

$$(1, r_i, 1, r_i) \ \text{ in block } \ i, \quad (1,1,1,1) \ \text{ in block } \ i+1 \tag{6}$$

when $0 \leq i < K - 1$. The last row instead uses

$$(1, r_{K-1}, 1, r_{K-1}) \ \text{ in block } \ K-1, \quad (1, T^2, T, T) \ \text{ in block } \ 0. \tag{7}$$

All omitted entries are zero. Every zero is a prescribed value, not a missing entry. The output bounds are $p = 2$ and $q = 2K$.

The row counts, in construction order, are $4K$, $K(K-1)$, $4K$, $K$ and $K$. They give Equation 2. The two blocks of every cycle row are distinct because $K \geq 3$. All entries are nonnegative integers.

## 4. Structure forced by a feasible decomposition

### 4.1. Saturation and pair sums

A nonzero tensor $a \otimes b$ is called a product vector. Under the usual coordinate identification, it is a rank-one matrix. In a decomposition satisfying Equation 1, associate the product vector $u_A \otimes w_A$ to row $A$, and $v_J \otimes \zeta_J$ to column $J$. Their pairing is $M_{A,J}$. Here $u_A, w_A$ are rows of $U, W$, while $v_J, \zeta_J$ are columns of $V, Z$.

**Lemma 2 (Saturation).** The first $N = 4K$ row product vectors form a basis of $\mathbb{R}^2 \otimes \mathbb{R}^{2K}$, and the column product vectors form its dual basis. A later row with coefficients $(a_J)_J$ has product vector $\sum_J a_J z_J$, where $z_J$ denotes the basis vector belonging to identity row $J$.

**Proof.** The leading identity gives a nonsingular pairing matrix of size $N$. The ambient tensor space has dimension exactly $N$. Both families are therefore bases and are dual. Pairing a later row with every dual basis vector determines each of its coordinates. This forces its entire product vector, not just a projection. Lower actual factor ranks cannot evade the argument, since the identity already requires the full ambient dimension. □

For two independent product vectors, their sum is a product vector exactly when they share a left or a right factor line. If both pairs of factors were independent, linear functionals on their spans would map the sum to a $2 \times 2$ identity matrix, contradicting rank one. The converse follows by distributing the tensor product.

Three or more independent product vectors with product-vector pairwise sums must share one factor line. To see this, fix two that share a left line and have distinct right lines. A third vector without that left line would have to share both distinct right lines, which is impossible. Thus it has the common left line. The argument with left and right exchanged is identical.

### 4.2. Synchronized selectors

**Lemma 3 (Two orientations).** In every feasible decomposition, there are independent left vectors $\alpha, \beta$ and a basis $b_0, c_0, \ldots, b_{K-1}, c_{K-1}$ of $\mathbb{R}^{2K}$ such that

$$z_{L_i} = \alpha \otimes b_i, \quad z_{R_i} = \beta \otimes c_i. \tag{8}$$

For each $i$, exactly one of the following orientations holds, with a nonzero real scalar $t_i$:

$$\begin{aligned} \text{Selected:} \quad & z_{X_i} = t_i \alpha \otimes c_i, \quad z_{Y_i} = t_i^{-1} \beta \otimes b_i, \\ \text{Unselected:} \quad & z_{X_i} = t_i \beta \otimes b_i, \quad z_{Y_i} = t_i^{-1} \alpha \otimes c_i. \end{aligned} \tag{9}$$

**Proof.** The pair-sum rows on the $L$ indices force a common factor line. A common right line would confine $K \geq 3$ independent vectors to a two-dimensional space. They therefore share a left line $\alpha$. The $R$ indices likewise share a left line $\beta$.

These two lines are distinct. Otherwise each pair $z_{L_i}, z_{R_i}$ would have the same left line and independent right factors. Its two common neighbors would be forced onto that left line as well. All $4K$ basis vectors would then lie in a space of dimension $2K$, a contradiction. Write the anchor vectors as in the statement.

Within each block, $b_i$ and $c_i$ are independent. If they were proportional, every common neighbor would have that right line: it cannot share both independent left lines. The four vectors would lie in a two-dimensional space. With both pairs of factor lines distinct, a common neighbor must occupy an off-diagonal tensor position. The two neighbors occupy opposite positions because they are independent. Initially write their nonzero multipliers as $t_i, s_i$.

All right vectors across blocks are independent. A proper span of the $2K$ right vectors would make the span of the $4K$ basis vectors smaller than $4K$. This proves the global independence needed for composition. In either orientation, the calibration row has determinant $1 - s_i t_i$ in the local $2 \times 2$ coordinates. Its decomposability forces $s_i t_i = 1$, proving Equation 9. □

Figure 1 depicts the two orientations after calibration. The shared left lines synchronize all selectors while the right basis keeps separate blocks independent.

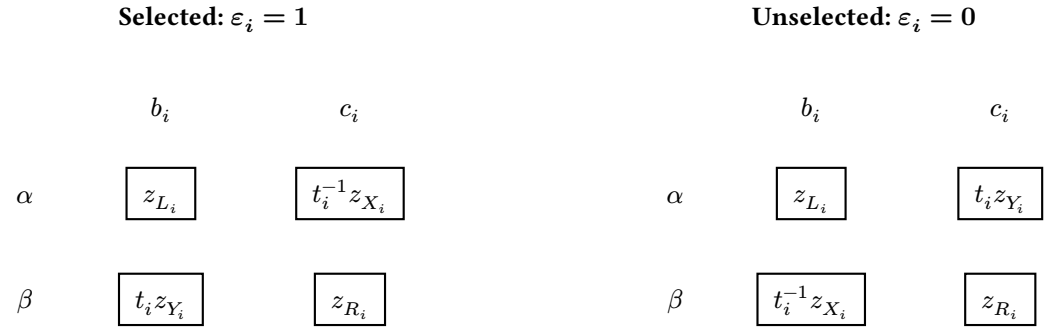



Figure 1: The two calibrated orientations of selector $i$. Rows denote the common left vectors $\alpha, \beta$; columns denote its independent right vectors $b_i, c_i$. Each box gives the scaled basis vector equal to the tensor product of its row and column labels. The off-diagonal positions exchange $X_i$ and $Y_i$. All scalars $t_i$ are nonzero; their signs are unrestricted.

## 5. Cycle equivalence and completeness

Let $\varepsilon_i$ be one for the selected orientation and zero otherwise. For block coefficients $(a, b, c, d)$ at indices $(L_i, R_i, X_i, Y_i)$, the local coordinate matrices supplied by Lemma 3 are

$$\begin{pmatrix} a & ct_i \\ \frac{d}{t_i} & b \end{pmatrix} \quad \text{(selected)}, \qquad \begin{pmatrix} a & \frac{d}{t_i} \\ ct_i & b \end{pmatrix} \quad \text{(unselected)}. \tag{10}$$

Define $h_i = \frac{1}{t_i}$ in the selected orientation and $h_i = t_i$ otherwise. Each pattern in Equation 5, Equation 6 and Equation 7 satisfies $ab = cd$ and has no zero entries. Its local block has rank one. The ratio of its second row to its first is

$$\begin{aligned} (1, 1, 1, 1) &\mapsto h_i, \\ (1, r_i, 1, r_i) &\mapsto r_i^{\varepsilon_i} h_i, \\ (1, T^2, T, T) &\mapsto T h_i. \end{aligned} \tag{11}$$

These formulas follow directly from the first column of Equation 10.

**Lemma 4 (Product cycle).** The constructed matrix has a real Hadamard decomposition at widths $(2, 2K)$ if and only if a subset of the item integers has product $T$.

**Proof.** Suppose a decomposition exists. By the global right-basis independence in Lemma 3, a cycle row has rank one only when its two nonzero blocks have the same left line. Applying Equation 11 gives

$$h_{i+1} = r_i^{\varepsilon_i} h_i \quad (0 \le i < K - 1), \quad T h_0 = r_{K-1}^{\varepsilon_{K-1}} h_{K-1}. \tag{12}$$

Every $h_i$ is nonzero. Multiplying these equations and cancelling gives

$$\prod_{i=0}^{K-1} r_i^{\varepsilon_i} = T. \tag{13}$$

No positivity of a factor or of $t_i$ has been assumed.

Conversely, suppose a subset satisfies Equation 13. Use the two standard left basis vectors for $\alpha, \beta$ and the standard right basis for all $b_i, c_i$. Set $h_0 = 1$ and compute the successive $h_i$ from Equation 12. Set

$t_i = \frac{1}{h_i}$ for a selected item and $t_i = h_i$ otherwise. The four vectors of each block in Lemma 3 are a scaled permutation of the standard tensor basis. Their dual basis also consists of product vectors.

Use this basis for the identity rows and its dual for the columns. Anchor and neighbor rows are product vectors because their summands share a factor. Calibration rows have zero local determinant. The cycle rows have equal left-line ratios by Equation 12, including the last row by the subset-product premise. Each resulting rank-one coordinate matrix factors into a left and a right vector. These provide $U, W$ row by row, while the product duals provide $V, Z$ column by column. Their pairings equal every prescribed entry of $M$. All constructed coordinates are rational. □

The cycle is drawn schematically in Figure 2. Lemma 1 followed by Lemma 4 proves the YES/NO equivalence in Theorem 1.




$$\prod_{i=0}^{K-1} r_i^{\varepsilon_i} = T, \quad h_0 \neq 0$$

Figure 2: The multiplicative constraint, shown as an unfolded cycle. $h_i$ is the nonzero left-line ratio of the calibrated block, and $\varepsilon_i$ records whether item $r_i$ is selected. Each ordinary edge multiplies the ratio by $r_i^{\varepsilon_i}$. The closing block prescribes $Th_0$. Intermediate edges are omitted schematically. Cancelling the nonzero initial ratio gives the subset-product equation.

## 6. Witness recovery and bit complexity

### 6.1. Recovery from arbitrary valid factors

The first row of $U$ belongs to identity index $L_0$. For each assignment item $i$, compute the determinant of its left-factor rows at $L_0$ and $X_i$. By Lemma 3, this determinant is zero exactly when item $i$ is selected:

$$\varepsilon_i = 1 \Leftrightarrow \det \begin{pmatrix} u_{L_0,1} & u_{L_0,2} \\ u_{X_i,1} & u_{X_i,2} \end{pmatrix} = 0. \tag{14}$$

This predicate is invariant under invertible changes of left coordinates and nonzero row gauges. It therefore applies to every valid real decomposition, not only to the witnesses constructed in Lemma 4. Read the first $2n$ item bits and assign variable $i$ true exactly when its true item is selected. Lemma 1 guarantees that exactly one item in each pair is selected and that the recovered assignment satisfies every clause.

### 6.2. The encoded zero test

For executable recovery, encode all factor entries in one real algebraic primitive element $\theta$. The witness supplies a nonzero integer polynomial $P$ and rational non-root endpoints $a < b$ whose interval contains exactly one distinct real root $\theta$ of $P$. Each factor entry is a dense rational coordinate polynomial evaluated at $\theta$. The polynomial $P$ may be reducible or have repeated roots. All degrees and coefficients are written explicitly. Rational numerators and denominators are integer literals; compressed exponents and symbolic expressions are excluded.

For the determinant polynomial $D$ in Equation 14, use the equivalence

$$D(\theta) = 0 \Leftrightarrow \gcd(P, D) \ \text{ has a root in } \ (a, b). \tag{15}$$

Every root of the gcd is a root of $P$, so the isolating premise makes the right-hand root count zero or one. A zero determinant gives $\gcd(P, 0) = P$; a constant gcd has no root. This handles localization to an irreducible factor without factoring $P$ or constructing another number field.

The executable computes the gcd up to scale as the last nonzero subresultant. For root counting, it clears denominators, takes the primitive integer part, and removes repeated factors using the gcd with

the derivative. It then computes the sign-corrected subresultant Sturm sequence. Subtracting its sign-variation count at $b$ from that at $a$, omitting zero evaluations, gives the required root count. These are standard exact polynomial operations [5]. The implementation uses SymPy's `Poly.subresultants` and `sturm_amv(method=1)` the latter computes sign-corrected subresultants as described in Section 2.4.3 of [6]. No feasibility solver runs inside either reduction map.

### 6.3. Bounds for both maps

Let $\ell$ be the source bit length and set $S = n + m + 2$. The first $n + m$ primes have magnitude $O(S \log S)$ by the classical explicit prime bounds [7]. Trial division generates them in polynomial time. The total number of literal occurrences is $3m$, so each item, $T$, and $T^2$ has $O(S \log S)$ bits. The dense target consequently has

$$O(S^3) \ \text{ entries and } \ O(S^4 \log S) \ \text{ bits.} \tag{16}$$

This is a loose encoding bound that charges the longest entry at every position. Schoolbook arithmetic and serialization give conservative bounds

$$O((\ell + 2)^8) \ \text{ forward bit operations,} \quad O((\ell + 2)^5) \ \text{ output bits.} \tag{17}$$

The rational witnesses in Lemma 4 also have polynomial encoding length: every $h_i$ is an item subproduct, with at most $O(S^2 \log S)$ bits under a loose bound. This statement concerns the constructed YES family only.

Let $H$ be the total serialized algebraic witness length and put $B = H + 2$. Literal rational parsing and dense coordinate encoding ensure that degrees and input coefficient heights are at most $B$. Each determinant has degree $O(B)$ and, after clearing denominators, coefficient height $O(B^2)$. A subresultant coefficient is a determinant of a Sylvester submatrix of order $O(B)$, giving height $O(B^3)$ for the normalized gcd. The square-free part retains this polynomial height; its sign-corrected subresultants have height $O(B^4)$. The subresultant size bounds underlying these steps are standard; see Theorems 2.1–2.4 of [5].

The controlled remainder procedure has $O(B^3)$ rational coefficient operations per sequence. Allowing $O(B^8)$ bits per intermediate numerator or denominator conservatively covers polynomial divisions on these bounded-degree, bounded-height inputs. Schoolbook rational arithmetic, normalization and endpoint evaluation therefore fit within $O(B^{24})$ bit operations per determinant. This deliberately loose bound applies to the subresultant procedure actually called. With $2n$ determinant queries and one field check, recovery takes

$$O((\ell + 1)(H + 2)^{24}) \ \text{ bit operations} \tag{18}$$

and outputs $O(\ell + 1)$ bits. No bound polynomial in the formula length alone is assumed for an arbitrary supplied witness.

The converse in Lemma 4 and the predicate in Equation 14 hold over all real factors. The executable guarantee concerns the finite algebraic encoding just defined. Since the feasibility equations have rational coefficients, a nonempty real solution set has a real-algebraic point by real closed field transfer. This fact does not assert polynomial witness length for the unrestricted problem.

Malformed source encodings map outside the target language. The canonical occurring-variable representation is polynomially obtainable from ordinary 3-SAT. Together, the construction, equivalence, recovery and bounds prove Theorem 1.

## 7. Relation to previous work and limits

The target is the factor-width formulation of Ciaperoni, Gionis and Mannila, Equation (4) of [1], with exact feasibility discussed in their Section 4.3 and the complexity question in Section 9. Our input matrices are integer and nonnegative; all factor variables remain unrestricted real. This distinction matters because imposing extra factor conditions changes the decision problem.

Identity factorizations and tensor-product representations are prior ingredients. Wertz, Vandaele and Gillis give an identity construction in Appendix B, Lemma 2 of [2]. Gillis and coauthors give a face-splitting and rank-one reshape formulation in Lemmas 2.3–2.4 of [3]. The contribution here is the synchronization of all selector blocks, calibration of their residual scales, and the multiplicative cycle.

Low-rank matrix signing and entrywise power factorization have their own hardness results [8]. In particular, Theorem 2.2 of that paper establishes hardness for prescribed magnitudes of a single factor. Two independent Hadamard factors admit additional magnitude freedom, so that theorem alone does not establish the present result. Generic identifiability of Hadamard–Hitchcock tensor decompositions [9] likewise does not decide exact feasibility of every matrix.

Primary-literature searches through 15 September 2026 found no matching hardness theorem for this unrestricted real two-factor model. Coverage included the papers above, Hadamard factorization aliases, subset product, product bases and generic Hadamard-rank results. Unindexed manuscripts, exhaustive citation closure and overlooked implications of general representation theorems remain outside that coverage. The claimed new construction requires independent human assessment of correctness and novelty.

## 8. Conclusion

Synchronized product-vector selectors convert exact subset product into Hadamard feasibility at widths 2 and $2K$. Their cycle gives a Karp reduction from 3-SAT, and their left-factor directions provide witness recovery for arbitrary valid factors. The result identifies a hardness boundary with one rank bound fixed. It does not establish strong NP-hardness, approximation hardness, membership of the unrestricted problem in NP, or completeness for the existential theory of the reals.

## Bibliography


[1] M. Ciaperoni, A. Gionis, and H. Mannila, "The Hadamard decomposition problem," *Data Mining and Knowledge Discovery*, vol. 38, pp. 2306–2347, 2024, doi: 10.1007/s10618-024-01033-y.

[2] S. Wertz, A. Vandaele, and N. Gillis, "Efficient algorithms for the Hadamard decomposition," *arXiv preprint arXiv:2504.13633*, 2025, [Online]. Available: https://arxiv.org/abs/2504.13633v2

[3] N. Gillis, S. Saha, S. Sicilia, and A. Vandaele, "Manifold-based Algorithms for the Hadamard Decomposition," *arXiv preprint arXiv:2605.28980*, 2026, [Online]. Available: https://arxiv.org/abs/2605.28980v1

[4] R. M. Karp, "Reducibility among Combinatorial Problems," in *Complexity of Computer Computations*, Plenum Press, 1972, pp. 85–103. doi: 10.1007/978-1-4684-2001-2_9.

[5] I. Z. Emiris, B. Mourrain, and E. P. Tsigaridas, "Real Algebraic Numbers: Complexity Analysis and Experimentation," in *Reliable Implementation of Real Number Algorithms: Theory and Practice*, in Lecture Notes in Computer Science 5045., Springer, 2008, pp. 57–82. doi: 10.1007/978-3-540-85521-7_4.

[6] A. Akritas, "Anna Johnson and Her Seminal Theorem of 1917," *Computer Tools in Education*, no. 2, pp. 13–35, 2016, [Online]. Available: https://faculty.e-ce.uth.gr/akritas/publications/100.pdf

[7] J. B. Rosser and L. Schoenfeld, "Approximate formulas for some functions of prime numbers," *Illinois Journal of Mathematics*, vol. 6, no. 1, pp. 64–94, 1962, doi: 10.1215/ijm/1255631807.

[8] N. Gillis, S. Saha, S. Sicilia, and A. Vandaele, "On the Complexity of Low-Rank Matrix Signing and Entrywise Power Matrix Factorization," *arXiv preprint arXiv:2607.04875*, 2026, [Online]. Available: https://arxiv.org/abs/2607.04875v2

[9] A. Oneto and N. Vannieuwenhoven, "Hadamard-Hitchcock decompositions: identifiability and computation," *arXiv preprint arXiv:2308.06597*, 2025, [Online]. Available: https://arxiv.org/abs/2308.06597v2

# A. Verification and reproducibility

## A.1. Evidence and its scope

The implementation was checked with Python 3.14.7, SymPy 1.14.0 and Z3 5.1.0. The manuscript uses Typst 0.15.1. The prepared suite tests six source formulas and performs eight recovery calls. A separately implemented verifier tests 29 distinct formulas, including 27 YES and two NO instances, and performs 109 recovery calls. Nine of these calls use algebraic gauges, including negative roots, cubic roots and reducible defining polynomials. Four focused calls reuse an existing source formula; they are not extra distinct instances. The generated cases use seed 13013 and two through four occurring variables. The largest tested target has 682 rows and 88 columns, with widths 2 and 44.

The verifier runs the actual candidate in fresh processes. It enumerates source assignments and validates the actual matrix against the constructed family before solving its subset-product coefficients by exact meet-in-the-middle enumeration. It independently builds target factors, checks every matrix entry, invokes recovery, and checks the returned assignment directly. Variants include non-diagonal coordinate changes, right-coordinate mixing, and signed row and column gauges.

The prepared target oracle instead uses Z3 Boolean-controlled linear prefix equations. Its 32 standalone three-item checks agree with exhaustive subset enumeration, with 16 YES and 16 NO cases. A separate 38-case arithmetic check compares sign-corrected subresultant root counts with a rational Sturm implementation. Scientific notation is rejected before coefficient or endpoint conversion. The defining quartic real-factor oracle is retained for tiny target examples; an unknown outcome is an error.

Both structural target NO oracles depend on Lemmas 2–4. Their finite agreement cannot establish those lemmas. A separate reviewer inspected the general proof and constructed further checks without importing the candidate. These included an unrestricted local real escape query, exact symbolic cycle identities, and full witnesses using repeated, reducible defining polynomials and higher-degree coordinate perturbations. The final review found no remaining repair requirement. This agent review and the finite checks do not replace independent human expert verification.

## A.2. Files and commands

All paths below are relative to the repository root. The candidate and contract are in `campaigns/hadamard-decomposition/work/`. The files `algorithm.py`, `proof.md`, `contract.md`, `preparation.md` and `verification.md` describe the rule and its checks. Current raw outputs are in `campaigns/hadamard-decomposition/rounds/013/`: `literal-rational-prepared-check.txt`, `literal-rational-check.txt`, `structural-oracle-check.txt` and `arithmetic-check.txt`. The final review is `campaigns/hadamard-decomposition/reviews/literal-rational/review.md`. Earlier evidence remains with its owning round or review.

From the repository root, run:

```
cd campaigns/hadamard-decomposition/work
python3 check.py --self-test
python3 check.py --oracle product-basis --candidate algorithm.py
python3 verify.py --candidate algorithm.py
typst compile manuscript.typ manuscript.pdf
```

The two candidate suites execute both maps. There are no configured solver, subprocess or campaign timeouts. The finite input domains are declared in `cases.json` and `verify.py`. To run the maps on supplied inputs:

```
python3 algorithm.py < source.json > target.json
python3 algorithm.py --extract < request.json > assignment.json
```

Here `source.json` is a formula with keys `n` and `clauses`. The second input has keys `source` and `target_solution`. Its target solution uses the four factor matrices plus `field`, exactly as specified in `contract.md`. Each coordinate is a dense list of literal rational coefficients. These two command modes share no process state.